

Response to Sheikh Sameer — AI Pipeline Integration Reference

LocalAI TV — Admin Dashboard Plan v1.2

To: Sheikh Sameer (AI Pipeline Engineer) **From:** LocalAI TV Architecture Team **Subject:** Answers to your 8 integration questions; confirmation of webhook contract **Date:** 14 May 2026 **Status:** All questions answered; improvements adopted into Plan v1.2

Note: This response is aligned with **Plan v1.2** which incorporates additional founder decisions: AI Pipeline migrates from Hostinger VPS to **AWS EC2 g4dn.xlarge (NVIDIA T4 GPU)** in Mumbai for 8-10× faster bulletin processing. Storage uses **AWS S3 (Mumbai)** with Cloudflare CDN for delivery. Scale target: 3,000 channels (pan-India) in 6 months. The integration contract described below remains valid; underlying infrastructure paths updated.

Dear Sameer,

Thank you for the **AI Pipeline Integration Reference** document. Your work has accomplished two important things:

1. **Confirmed the architecture is sound** — your pipeline already follows the right patterns (async Celery, provider abstraction, concurrency controls).
2. **Pinned down the integration contract** — your 8 open questions have forced us to specify the exact interface, which prevents ambiguity during build.

This document answers each of your questions definitively and approves all your improvement suggestions.

Confirmation: Your Pipeline Architecture Is Already Right

Your document revealed important details we need to align with:

Pipeline element	Your existing implementation	Our acknowledgment
Intake layer	FastAPI (returns fast, no inline processing)	✅ Webhook contract aligned
Job execution	Celery distributed tasks	✅ Admin won't run anything inline
Queue	(existing — and per Gnana's doc, RabbitMQ)	✅ Confirmed
LLM provider	<code>GeminiHandler</code> via <code>get_llm_handler(location)</code>	✅ Admin sends <code>target_channel</code> for routing
LLM fallback	OpenAI handler when <code>GEMINI_API_KEY</code> unavailable	✅ Sensible default
TTS provider	<code>tts_handler_gcp.py</code> with <code>MAX_TTS_CONCURRENCY</code>	✅ Concurrency already production-grade
Channel-level TTS routing	<code>TTS_PROVIDER_<CHANNEL></code> env var	✅ Admin sends <code>target_channel</code> in payload
Output	MP3 + FFmpeg-assembled MP4 in <code>ai-processed/</code>	✅ Folder structure agreed
Default model	<code>gemini-2.0-flash</code>	✅ No change requested

This means the integration is mostly a thin webhook adapter on your side, not a pipeline rewrite.

Answers to Your 8 Open Questions

Section 1 — Core Flow

Q1.1: Should the webhook return a `job_id` so Admin Dashboard can poll for status? What is the expected callback format?

Answer: YES — both polling AND callback are supported.

Webhook response (must return within 200ms):

```
{
  "status": "queued",
  "ok": true,
  "content_id": "abc-123-def",
  "approved_version": 1,
  "job_id": "celery-task-id-abc123",
  "idempotency_status": "new",
  "accepted_at": "2026-05-14T10:30:00Z",
  "estimated_completion_at": "2026-05-14T10:32:00Z"
}
```

Possible `idempotency_status` values:

- `new` — first time seeing this idempotency_key
- `already_queued` — duplicate request, job exists

- `already_completed` — duplicate request, already processed
- `rejected` — payload validation failed (with `errors` array)

Completion callback format (you POST to us):

```
POST https://admin.localaitv.com/api/webhook/ai-processed
Content-Type: application/json
Authorization: Bearer <shared_callback_token>
X-LocalAITV-Idempotency-Key: <same key from original webhook>

Body:
{
  "content_id": "abc-123-def",
  "approved_version": 1,
  "job_id": "celery-task-id-abc123",
  "status": "ready_for_bulletin", // or "failed_retryable" or "failed_final"
  "ai_output_url": "https://content.localaitv.com/ai-processed/AP/Kurnool/Kurnool-Urban/new",
  "thumbnail_url": "https://content.localaitv.com/ai-processed/AP/Kurnool/Kurnool-Urban/new",
  "duration_seconds": 47,
  "processed_at": "2026-05-14T10:35:00Z",
  "error_message": null, // populated if status is failed_*
  "metrics": {
    "gemini_time_ms": 1240,
    "tts_time_ms": 3820,
    "ffmpeg_time_ms": 12500,
    "total_time_ms": 17560
  }
}
```

Fallback polling endpoint (Admin → AI):

```
GET /api/v1/jobs/{job_id}
Returns: { status, progress_percentage, current_stage, error_message }
```

Admin Dashboard uses callback by default; polls only if callback not received within 10 minutes.

Q1.2: What is the SLA for the Celery worker to pick up and start processing? Is there a max queue depth threshold?

Proposed SLAs (open to your team's adjustment):

Metric	Target	Alert threshold
Webhook receipt → Celery pickup	<5 seconds at normal load	>30 seconds
Celery pickup → first AI call	<2 seconds	>10 seconds
Total pipeline (approve → ready_for_bulletin)	2–5 minutes typical	>10 minutes triggers concern
Queue depth	<100 pending jobs	>500 pages on-call
Failed jobs in last hour	<1%	>5%
Worker count alive	=N (configured workers)	<N triggers alert

Recommended implementation:

- **Flower** (Celery monitoring UI) — exposes queue metrics
- Export to **Grafana Cloud** (free tier) for dashboards + alerting
- **PagerDuty / Slack** integration when thresholds breach

Please confirm these targets are realistic given your current worker pool sizing.

Q1.3: If the AI Pipeline callback to Admin Dashboard fails (network error, timeout), how many retries are attempted and what happens after exhaustion — is the broadcast slot held or released?

Answer:

Retry policy (your side):

- **5 attempts with exponential backoff:** 1s → 5s → 30s → 5min → 30min
- After 5th failure: write to your local dead-letter queue + Sentry alert + Slack notification
- Maintain the processed output in `ai-processed/` storage — it must not be lost

Admin Dashboard side watchdog:

- `pg_cron` runs every 5 minutes
- Finds content stuck in `status = 'ai_processing'` for >30 minutes with no callback
- Marks `status = 'callback_lost'`
- Notifies super admin: *"Content X is processed in your storage but callback was never received. Please verify in ai-processed/{path} and manually confirm."*

Broadcast slot behavior:

-  **Slot is HELD (reserved)** until callback received or super admin manually clears
-  **Slot is NOT auto-released** — better to show "standby content" filler than air an unconfirmed/missing item
- Super admin has a "Force clear slot" action that releases it for fresh content

Rationale: Airing without confirmation could broadcast wrong content or empty audio. Holding the slot is safer.

Q1.4: Is the `ai-processed/` output path configurable per environment (dev/staging/prod), or is it hardcoded?

Answer: CONFIGURABLE via environment variables — 12-factor compliant.

In v1.2, storage is **AWS S3 (Mumbai region)** with Cloudflare CDN in front for delivery:

```
# Dev
AWS_S3_BUCKET=localaitv-content-dev
AWS_REGION=ap-south-1
AI_OUTPUT_BASE_URL=https://content-dev.localaitv.com/ai-processed # via Cloudflare CDN

# Staging
AWS_S3_BUCKET=localaitv-content-staging
AWS_REGION=ap-south-1
AI_OUTPUT_BASE_URL=https://content-staging.localaitv.com/ai-processed

# Production
AWS_S3_BUCKET=localaitv-content-mumbai
AWS_REGION=ap-south-1
AI_OUTPUT_BASE_URL=https://content.localaitv.com/ai-processed # production CDN
```

Why S3 + Cloudflare CDN (not direct delivery from S3):

- Without CDN: S3 egress fees at 3,000-channel scale = ₹5-10 crore/month
- With Cloudflare CDN: ~90% cache hit ratio = ~₹50L/month at peak
- Same architecture, very different unit economics

Your AI Pipeline uploads directly to S3 (using AWS SDK with `boto3`). The mobile app fetches via Cloudflare CDN URL (which proxies to S3 on cache miss).

Path convention (per Gnana's proposal — adopted):

```
ai-processed/<state>/<district>/<constituency>/<category>/<content_id>/
├─ final.mp4
├─ headline.mp3
├─ thumb.jpg
└─ meta.json
```

For bulletin assembly, your existing build worker uses a separate path:

```
ai-processed-bulletins/<state>/<district>/<constituency>/bul_<YYYYMMDD_HHMMSS>/
├─ final_bulletin.mp4
├─ bulletin_manifest.json
├─ thumbnail.jpg
├─ segments/item_001.mp4 (etc.)
└─ metadata.json
```

Section 2 — Webhook Trigger Rules

Q2.1: Can already-approved content be edited and re-approved? Does a second webhook fire, and does the pipeline cancel/supersede the previous job?

Answer for v1: NO — once approved, content is locked from further edits.

Recovery path if admin discovers a mistake post-approval but pre-broadcast:

1. Super admin clicks "Cancel processing" on the item
2. Admin Dashboard sends: DELETE /webhook/cancel-job/:job_id
3. AI Pipeline (your side):
 - If job not yet picked up by Celery: remove from queue → 200 OK
 - If job is currently processing: send Celery SIGTERM, task cleans up → 200 OK
 - If job already complete: too late → 410 Gone
4. Content moves back to status='pending' in Admin Dashboard
5. Admin can edit and re-approve normally → new webhook with approved_version=2

Phase 2 enhancement: Native re-approval flow with version supersession. Deferred until we observe real need.

Q2.2: Is there a validation schema for the webhook payload? What happens if required fields are missing — does the pipeline reject with 4xx or silently skip?

Answer: Strict JSON Schema (Pydantic). Pipeline returns 400 with field-level errors. NEVER silently skip.

Pydantic model:

```

from pydantic import BaseModel, Field
from typing import Literal, Optional
from datetime import datetime

class MediaItem(BaseModel):
    type: Literal["photo", "video"]
    url: str
    size_bytes: int
    duration_seconds: Optional[int] = None
    thumb_url: Optional[str] = None
    sha256: str # for dedup

class ApprovedContentWebhook(BaseModel):
    # Identifiers
    content_id: str
    approved_version: int = 1
    idempotency_key: str
    job_id: Optional[str] = None # AI Pipeline generates if missing

    # Geography (national OR state required minimum)
    geography_level: Literal["national", "state", "district", "constituency"]
    state: Optional[str] = None
    district: Optional[str] = None
    constituency: Optional[str] = None

    # Content
    category: Literal["news", "birthday", "marriage_day", "upcoming_marriage",
                     "event", "shopping", "car_sale", "rental", "job"]
    headline: str = Field(min_length=4, max_length=300)
    description: str = Field(min_length=20, max_length=3000)
    location: Optional[str] = None
    form_data: dict = Field(default_factory=dict)

    # Media
    media_urls: list[MediaItem] = Field(min_length=1, max_length=3)

    # Author + audit
    author_id: str
    author_role: Literal["citizen", "admin", "master_admin", "super_admin"]
    approved_by: str
    approved_at: datetime

    # Channel routing
    target_channel: str # e.g. "kurnool-tv" → controls TTS_PROVIDER_<CHANNEL>

    # Processing mode
    processing_mode: Literal["ai_pipeline", "raw_video", "notebooklm_generated"]

```

On invalid payload:

```

HTTP 400 Bad Request
{
  "error": "validation_failed",
  "errors": [
    {"field": "headline", "issue": "must be at least 4 characters, got 2"},
    {"field": "media_urls", "issue": "must contain at least 1 item, got 0"},
    {"field": "category", "issue": "must be one of: news, birthday, ..."}
  ]
}

```

Admin Dashboard logs validation failures to Sentry and notifies super admin.

Q2.3: Who is responsible for filtering out non-approved states before the webhook fires — Admin Dashboard or AI Pipeline? Is there a duplicate guard?

Answer: Admin Dashboard filters. AI Pipeline trusts the sender. Duplicate guard via idempotency key.

Responsibility split:

Check	Owner	Implementation
Only fire webhook when status='approved'	Admin Dashboard	RLS + application logic
Block double-firing within Admin	Admin Dashboard	<code>UPDATE content SET webhook_sent_at=NOW() WHERE ... AND webhook_sent_at IS NULL</code> (race-safe)
Detect duplicate received webhooks	AI Pipeline	Via <code>idempotency_key</code> (your <code>content_id + approved_version</code> format)
Block double-processing in Celery	AI Pipeline	<code>Task.apply_async(task_id=idempotency_key)</code> is naturally idempotent

Idempotency lifecycle:

1. Admin generates: `idempotency_key = "abc-123_v1"`
2. Admin sends webhook with this key
3. AI Pipeline checks: has this key been seen before?
 - First time: enqueue Celery task with `task_id=key`, `status="new"`
 - Already queued: return 200 OK, `status="already_queued"`
 - Already completed: return 200 OK, `status="already_completed" + ai_output_url`
4. Even if Admin retries due to network error, AI Pipeline never double-processes

Q2.4: What is the maximum size of `media_urls` array? Are there limits on media file sizes or formats accepted?

Answer:

Limit	Value	Rationale
Max items in <code>media_urls</code>	3	Matches mobile app hard limit
Photo file size	≤ 10 MB	Mobile photo upper bound
Video file size	≤ 100 MB	~2 min mobile video
Total payload bytes	≤ 200 MB	Practical for FFmpeg processing
Photo formats	JPG, PNG, WebP	Standard mobile output
Video formats	MP4 (H.264/AAC), MOV, WebM	iOS + Android native
Audio (rare)	MP3, M4A	For voice-only ads
Signed URL expiry	1 hour	Long enough for Celery download, short for safety

Handling unavailable/corrupt files:

AI Pipeline downloads media from signed URL
→ 404 / corrupted file:
Return HTTP 422 Unprocessable Entity to callback
Admin moves content to status='media_invalid'
Admin can re-request upload or reject the item

Improvement Suggestions — All Adopted

Your "Still Useful to Improve" list is fully approved:

Improvement	Implementation	Owner	Cost
Centralized per-stage timing logs	OpenTelemetry spans for: webhook receipt → Celery pickup → Gemini → TTS → FFmpeg → callback. Export to Sentry.	Your team	₹0
Provider-level success/failure metrics	Counters per provider per stage. Export to Grafana Cloud (free) or PostHog (free 1M events/mo).	Your team	₹0
Gemini fallback policy (explicit)	If Gemini fails 3× with backoff → fall back to OpenAI handler. If both fail → notify admin for manual script entry.	Your team	(already coded, just policy doc)
GCP TTS fallback policy (explicit)	If GCP TTS fails 3× → fall back to Sarvam. If both fail → notify admin.	Your team	(already coded)
Separate Celery queue for high-volume TTS	<code>tts_high_volume</code> queue for batch regenerations; <code>tts_realtime</code> for live broadcast prep	Your team	Celery config

These improvements automatically also address Abishek's monitoring (#9) and failure recovery (#1) concerns. Synergy across teams.

What's New for Your Side in v1.1 (Implementation Effort)

Per your "Still Useful to Improve" + Gnana's "Still Required for Full Admin Dashboard Contract" list combined:

Task	Estimated effort
<code>POST /webhook/content-approved</code> endpoint (FastAPI)	~1 day
HMAC signature verification	~2 hours
<code>content_id + approved_version</code> idempotency check	~2 hours
R2 signed-URL media download adapter	~1 day
Output upload to <code>ai-processed/</code>	~half day
<code>POST /api/webhook/ai-processed</code> callback to Admin	~half day
Callback retry mechanism (5 attempts)	~half day
Direct-publish lane registration (admin/notebooklm)	~half day
R2 lifecycle cleanup policy	configuration only
OpenTelemetry instrumentation	~1 day
Fallback policy documentation + alerts	~half day
Separate Celery queue for TTS bulk	~half day
TOTAL	~1 week for your team

Your existing pipeline (Gemini, TTS, FFmpeg, build worker) requires **zero changes**. The work is purely the integration adapter.

Coordinating with Other Engineers

Three engineers have reviewed the plan:

- **Abishek** (full-stack) → flagged production reliability gaps; our v1.1 includes pg_boss queues, idempotency, observability, security hardening.
- **You** (AI engineer) → defined the integration contract; your 8 questions are answered in this letter.
- **Gnana** (AI engineer) → extended the contract with concrete details (idempotency format, retention policy, bulletin output structure). His 9 questions also answered.

Your contribution and Gnana's are complementary — you defined the *interface*, Gnana detailed the *implementation*. Together they form the complete spec.

Final Notes

Your document was professional, structured, and asked exactly the right questions. The integration contract is now locked in v1.1.

If anything in this response needs further clarification — especially the Pydantic schema, idempotency lifecycle, or retry/dead-letter policy — please reply with specific points and we'll iterate.

With respect,

LocalAI TV Architecture Team LocalAI Media Network Pvt Ltd CIN: U63910KA2025PTC212593
Hyderabad, India

Plan v1.1 (separate document) contains the full integration contract reflected above.